Красно-чёрное дерево

Красно-черное дерево (КЧ дерево) - сбалансированное дерево поиска, операции над которым выполняются за $O(\log n)$. Также КЧ дерево удовлетворяет следующим свойствам:

1. *Любой* узел КЧ дерева должен быть либо красным, либо чёрным.
2. Корень КЧ дерева всегда *чёрный*.
3. Количество *чёрных* вершин при спуске от любого узла до листа одинаково для всех узлов на одной высоте.
4. Оба потомка красного узла **всегда** чёрные

$h(X)$ - высота дерева от корня до узла X.
$bh(X)$ - количество чёрных узлов на пути от корня до узла X.

Основные операции над КЧ-деревьями

Поиск

Поиск в КЧ дереве реализован аналогично любому бинарному дереву.

Алгоритм поиска в КЧ дереве:

1. Пусть ищем число x.
2. Запускаем рекурсивный поиск от корня для числа x.
3. Сравниваем значение узла y с числом x.
    1. Если $x < y$, то переходим в левого потомка y. Возвращаемся в п.3.
    2. Если $x > y$, то переходим в правого потомка y. Возвращаемся в п.3.
    3. Если $x = y$, то x найдено.

Сложность: $O(h)$, где h - высота дерева.

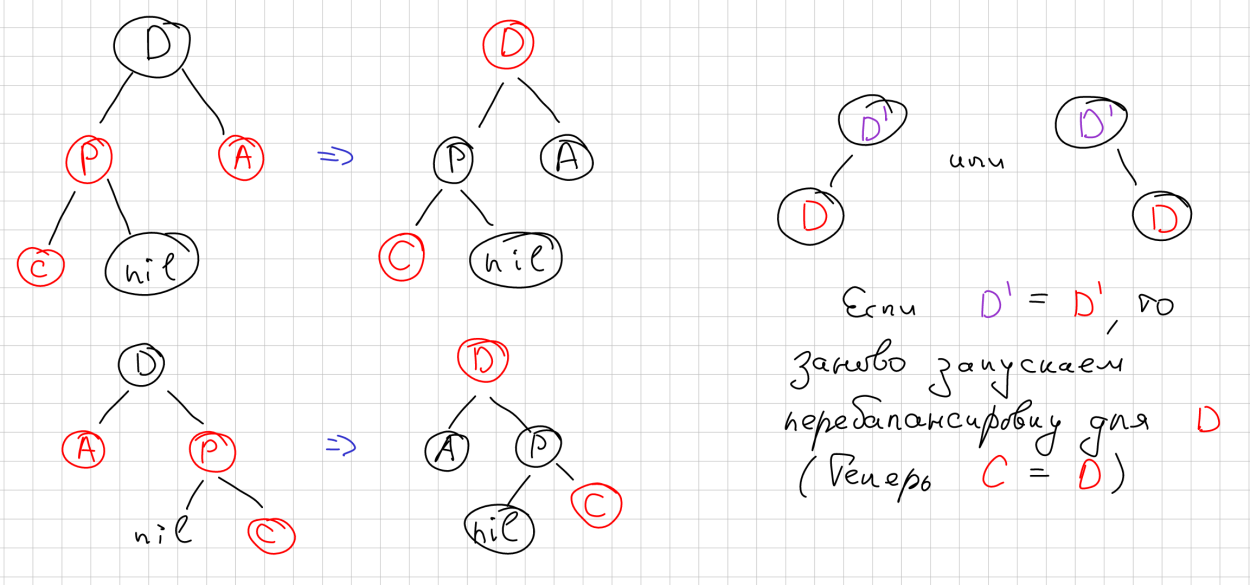
Вставка

В дерево **ВСЕГДА** вставляется **красная** вершина, при этом могут быть нарушены свойства 2 и 4, которые можно достаточно легко исправить.

Допустим дерево пустое. Вставляем красную вершину, получается, что нарушается свойство 2. Т.к. узлов в дереве больше нет, то просто перекрашиваем узел в чёрный, ни одно из свойст КЧ дерева а таком случае не будет нарушено.

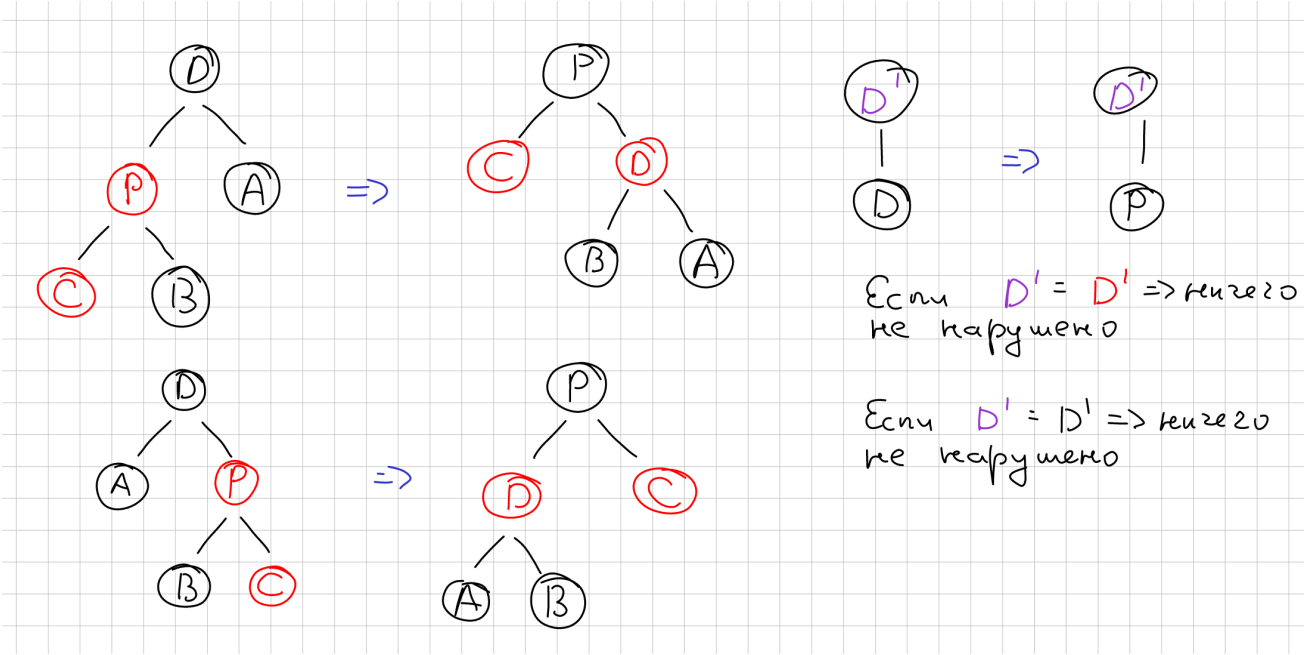
Допустим имеется красный узел P. Вставляем ему в потомка красный узел C. Тогда нарушается свойство КЧ-дерева. Ввёдем обозначение для дополгительных рассматриваемых узлов: B - брат C и потомок P; D - предок P; A - брат P и потомок D. Рассмотрим 3 ситуаций, которые возможны в таком случае, а также как их исправлять:

1. C, P, A - красные, D - чёрный, $B = $ `nullptr` (чтобы до вставки дерево было сбалансировано): В таком случае дед D перекрашивается в красный, оба его потомка P и A перекрашиваются в чёрный.
   - Если после перекраски деда D в красный он оказался корнем, то перекрашиваем его в чёрный. Чёрная высота $bh(x)$ увеличится на 1 для всех узлов.
   - Если дед D не является корнем, то его предок мог быть красным. В таком случае запускаем проверку для деда D (дед становится C).

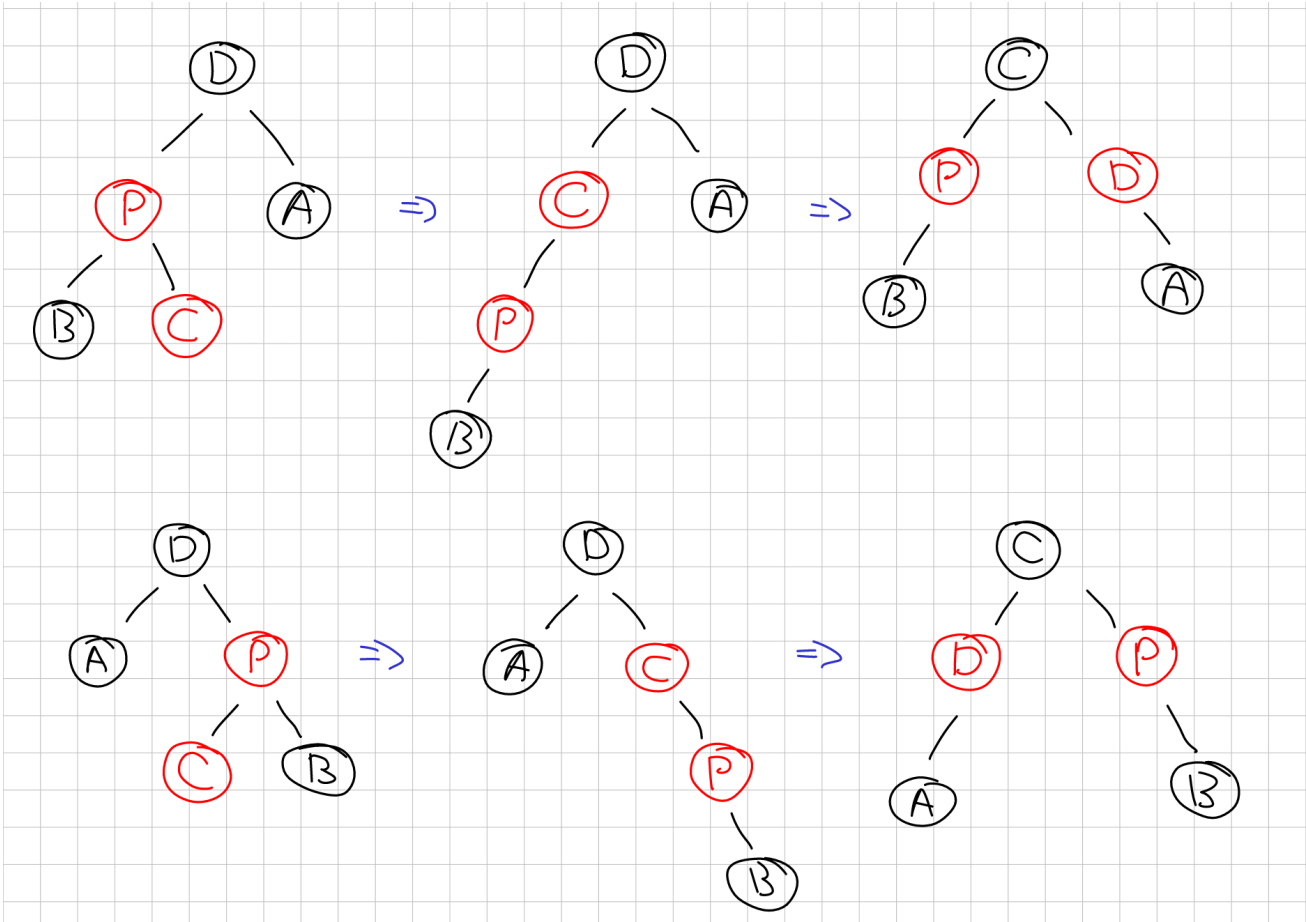


2. C, P - красные, A, D, B - чёрные, причём C - внешний узел, $A = B = $ `nullptr` (чтобы до вставки дерево было сбалансировано): В таком случае делаем поворот относительно узлов P и D и меняем их цвета на противоположные. Получаем P - чёрный дед; C и D - красные потомки P; B - внутренний чёрный потомок D; A - внешний чёрный потомок. Цвет предка D до перестановок не влияет, т.к. вместо

чёрного узла поставили новый чёрный узел.



3. C, P - красные, A, D, B - чёрные, причём C - внутренний узел, $A = B =$ `nullptr` (чтобы до вставки дерево было сбалансировано): В таком случае делаем поворот относительно узлов P и C. Получается структура п.2.



Сложность: $O(h)$, где h - высота дерева.

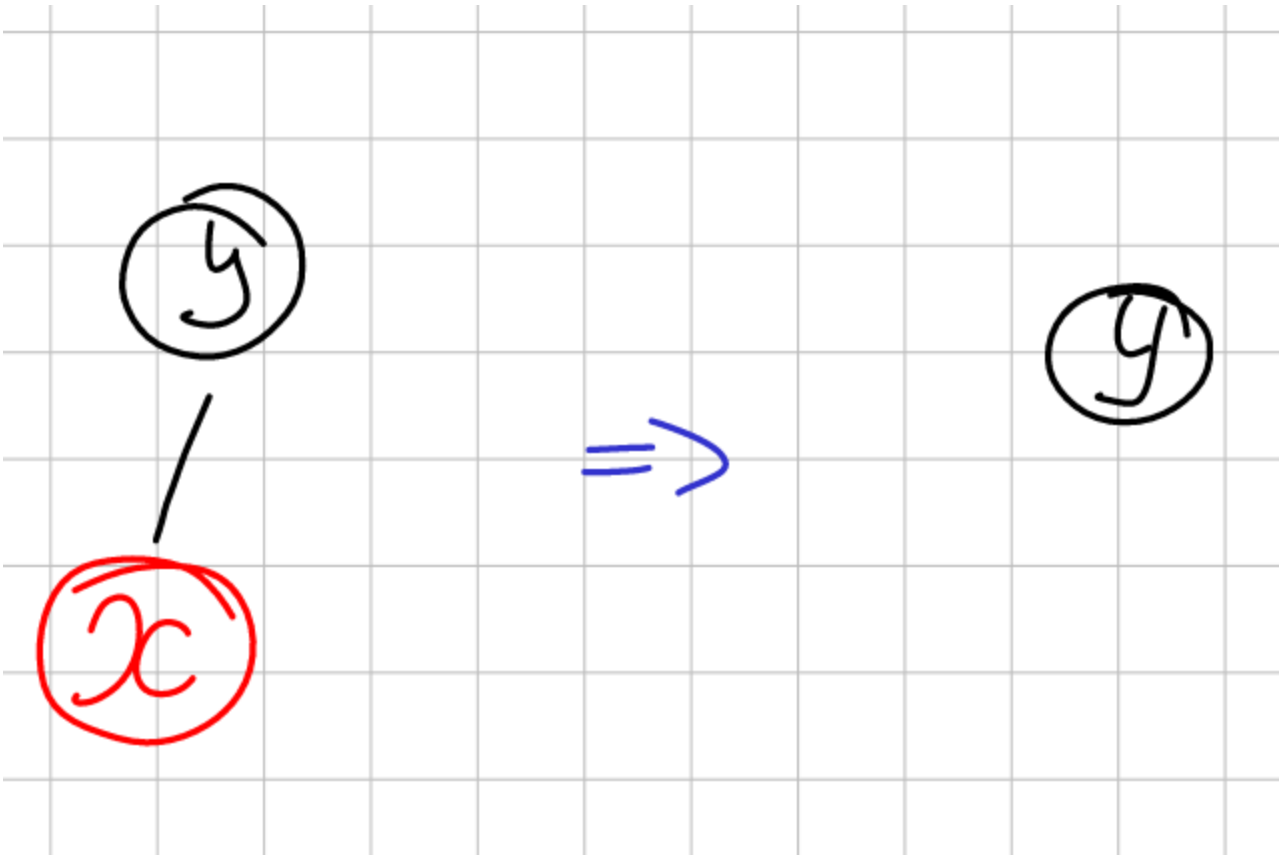
Доказательство: Чтобы найти, куда вставлять, необходимо найти позицию. Поиск всегда выполняется за $O(h)$. После вставки возможна балансировка, в которой повороты

выполняются за константу, но она может продолжаться, пока от узла не дойдём до корня, что составит h балансировок. Итого: $O(h + h) = O(2h) \approx O(h)$.

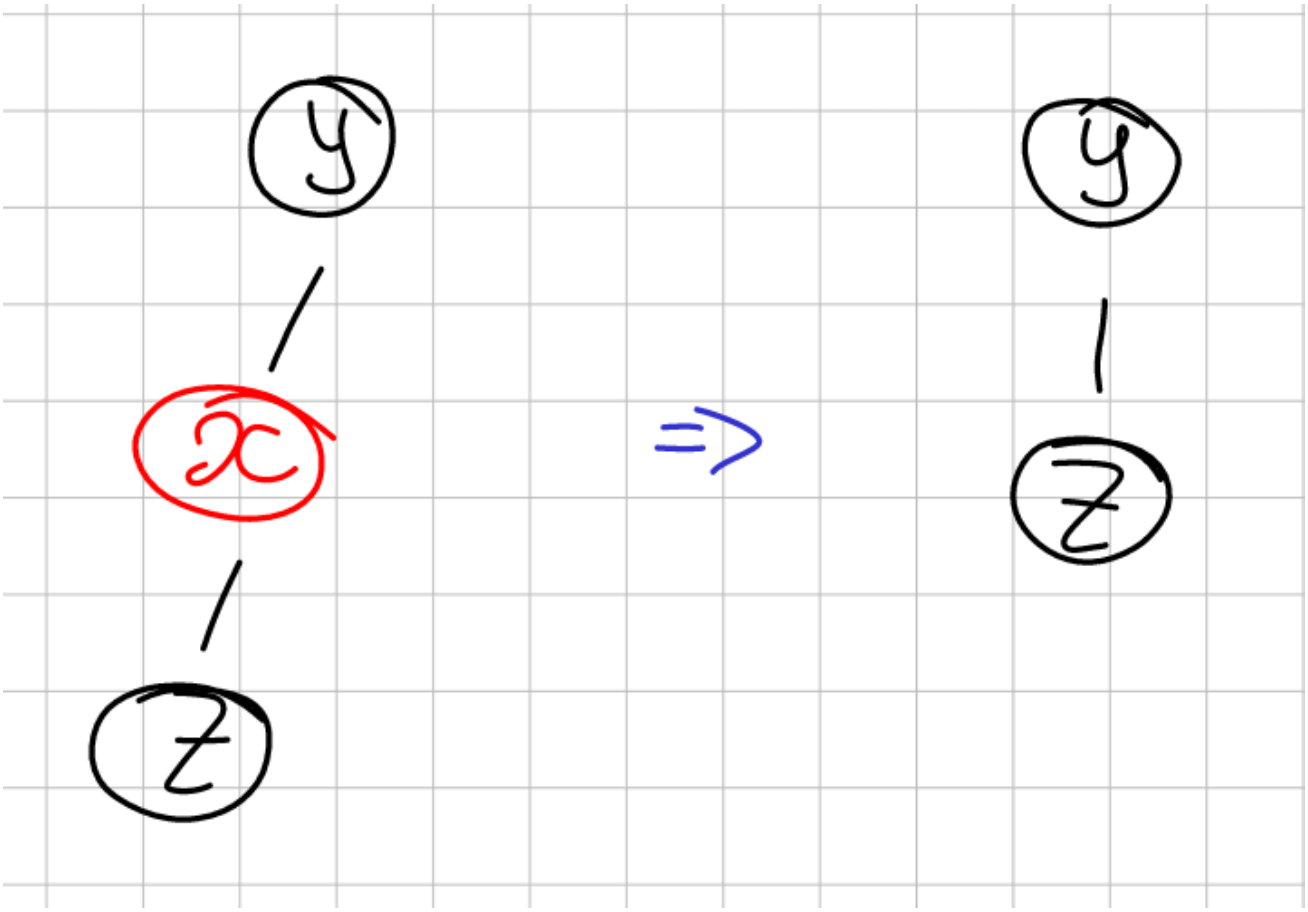
Удаление

Возможные ситуации при удалении *красного* узла X (X не может быть корнем):

1. Узел X не имеет потомков - прости затираем его.

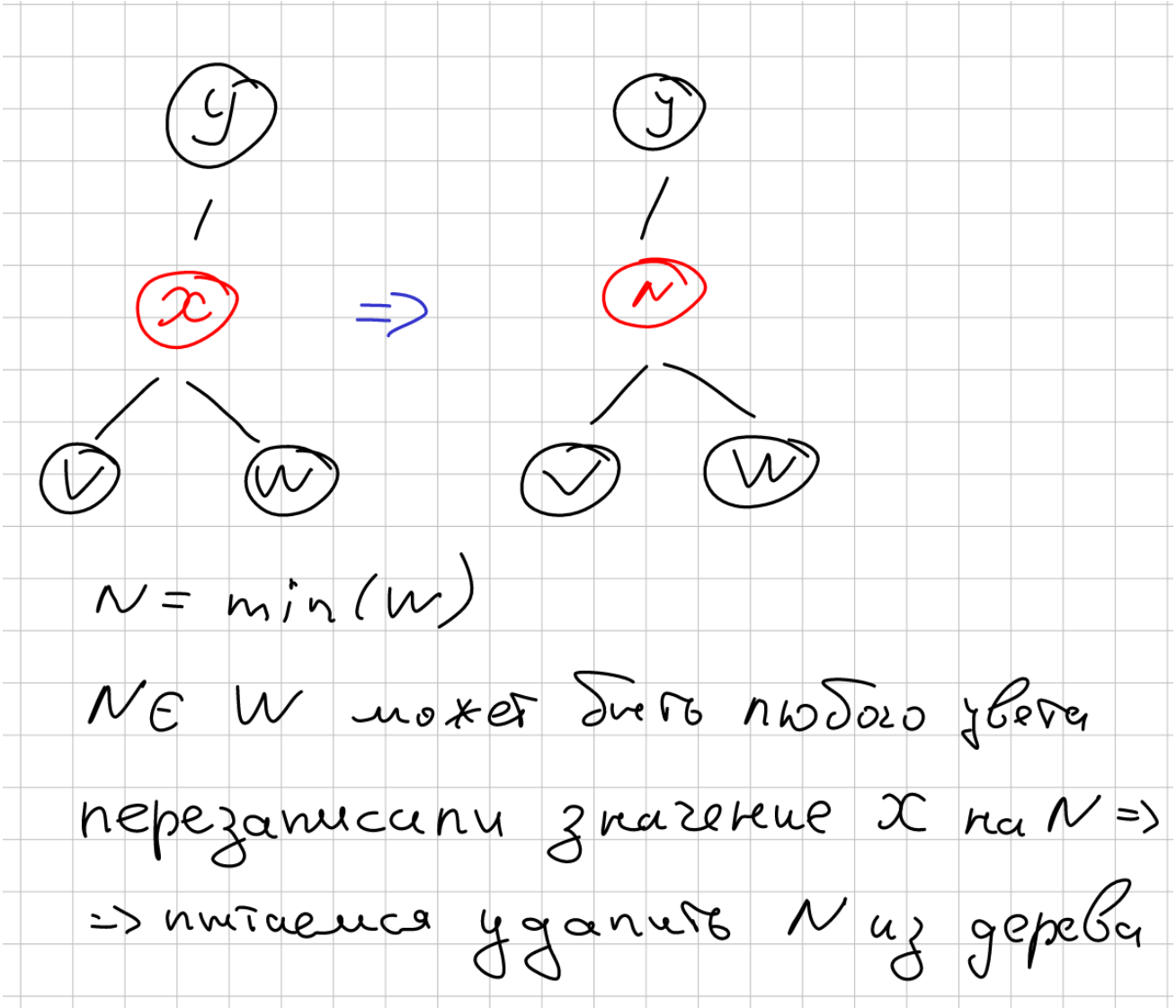


2. Узел X имеет предка Y и одного потомка Z, тогда потомков Y становится Z, а X просто затирается.



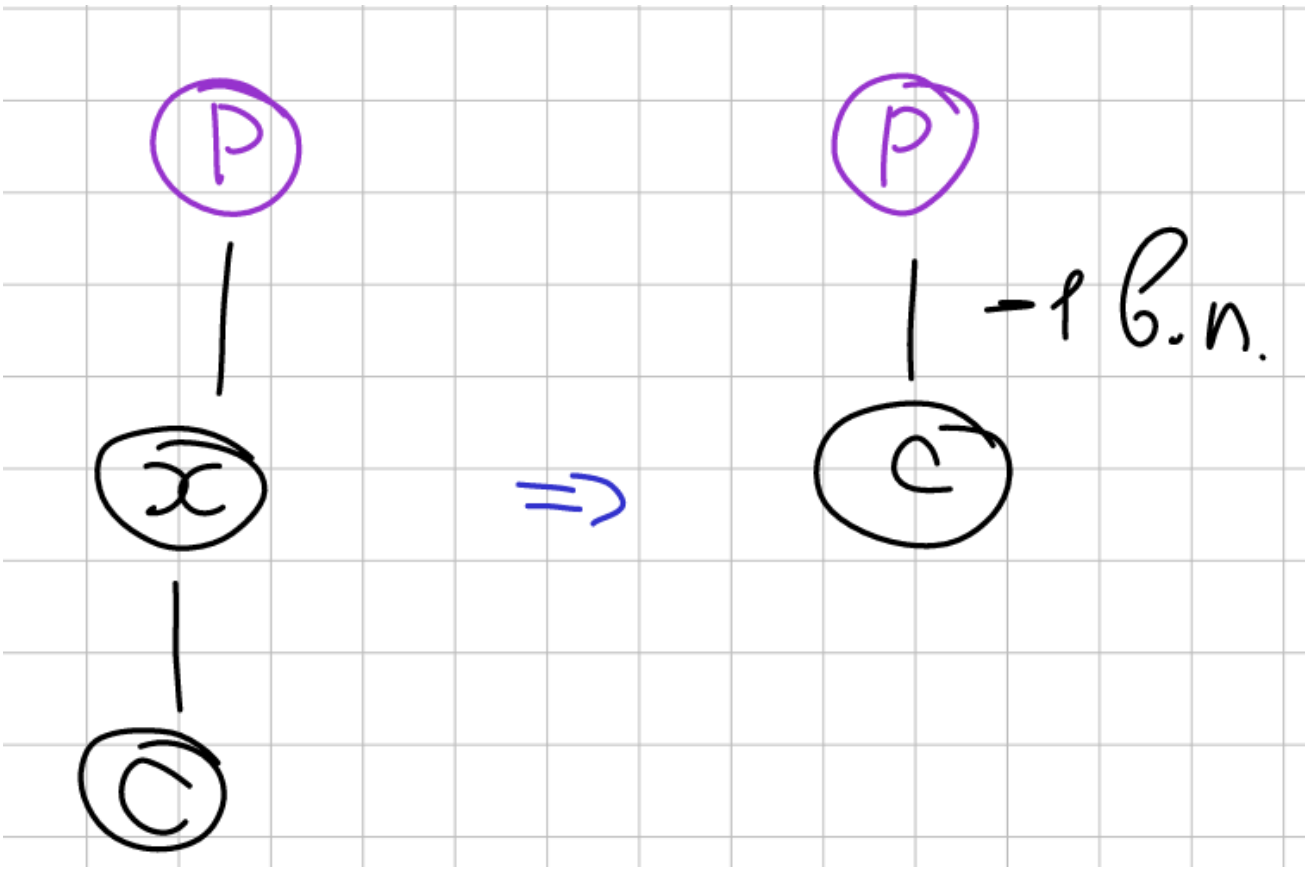
3. Узел X имеет двух потомков. Находим в правом потомке узел N с минимальным значением. Перезаписываем значение узла N в узел X, сохраняя изначальный цвет

узла X. Пытаемся удалить узел N.



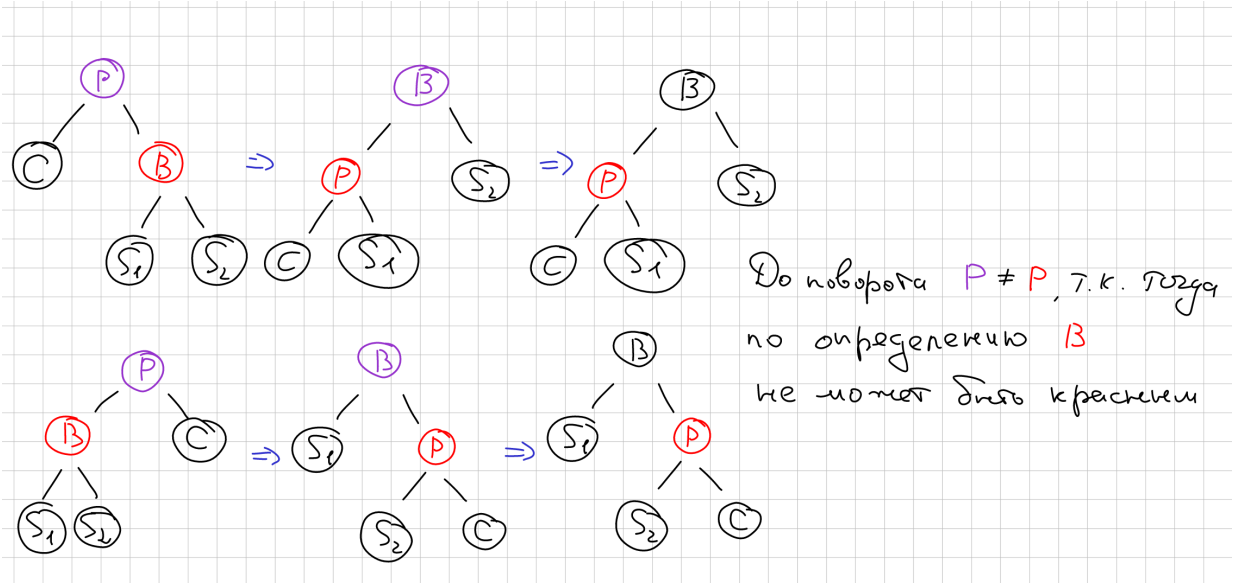
Возможные ситуации при удалении *чёрного* узла X:

1. X - корень дерева и у него нет потомков - затираем его - дерево пустое.
2. X - корень дерева и у него один потомок - затираем корень - независимо от цвета потомка он становится корнем и перекрашивается в чёрный.
3. X - узел (возможно корень) и у него два потомка - идём в правого потомка и находим в нём узел N с минимальным значением - перезаписываем значение узла N в X, сохраняя цвет X - пытаемся удалить узел N там, откуда было взято его значение.
4. X - узел и у него 1 *красный* потомок C (P - предок X). Тогда X затирается, потомком P становится C сохраняя *чёрный* цвет X. Цвет предка P ни на что не влияет, т.к. $bh(x)$ для него не изменится.
5. X - узел и у него либо нет потомков (два чёрных `nullptr`), либо один потомок C чёрного. Тогда X затирается, C становится потомком P (P был предком X) вместо X.

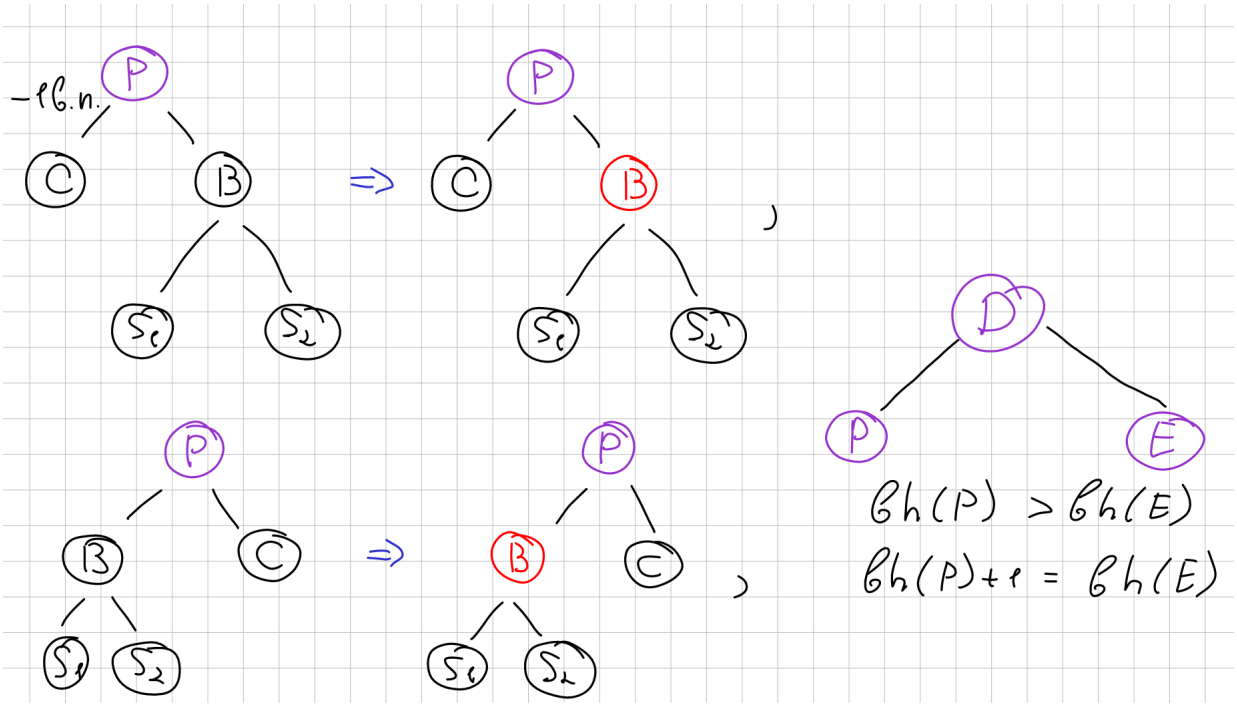


Получается, что для узла P черная высота через C меньше чёрной высота через B (B - второй потомок P). В зависимости от цвета B выполняются следующие балансировки:

1. Если B красный и у него два чёрных потомка S_1 и S_2 (возможно `nullptr`), то делаем поворот относительно P и B, после чего братом C становится чёрный $S1$ или S_2. Свели к ситуациям из п.5.2 - п.5.4.

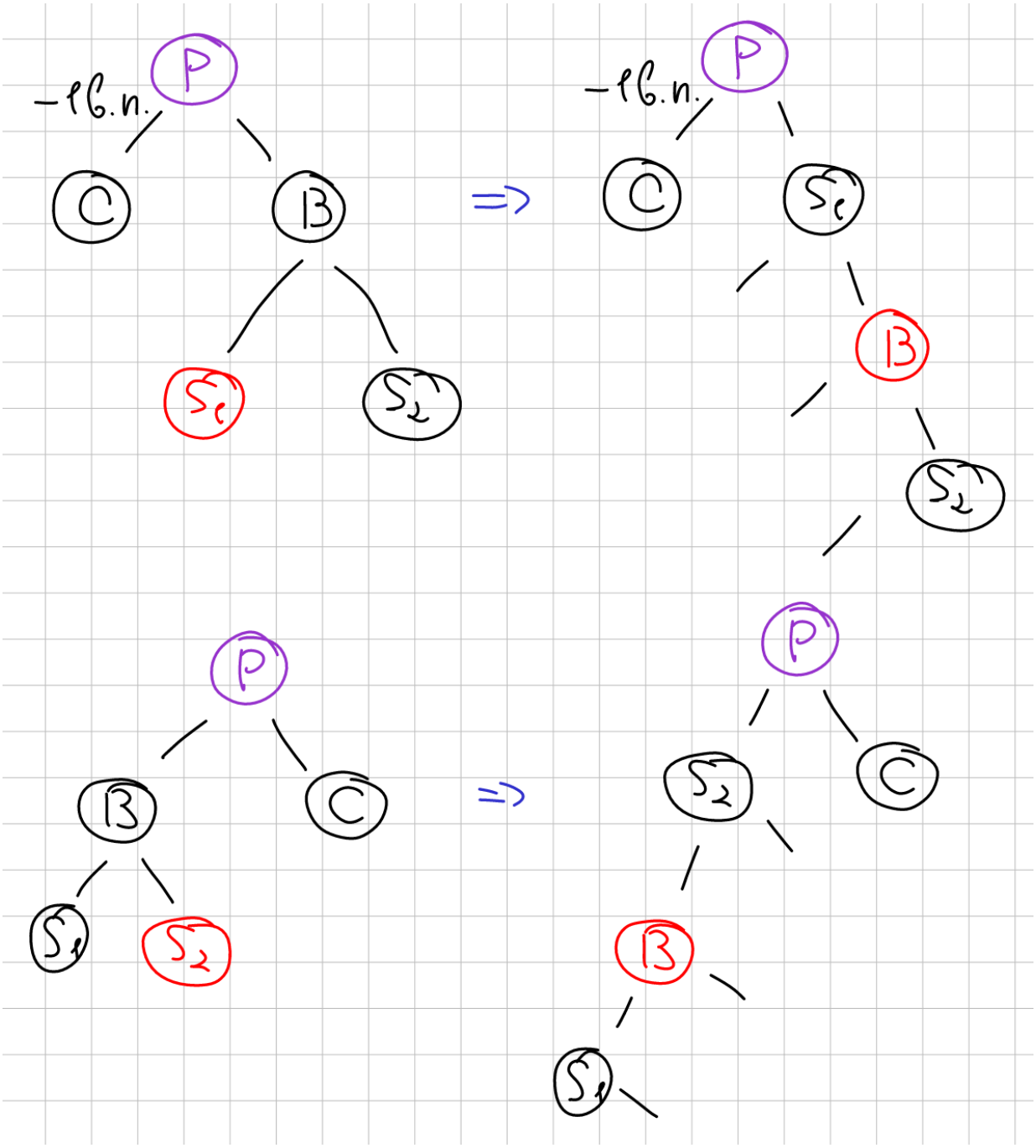


2. Если B чёрный и два его потомка чёрных (возможно оба `nullptr`). Тогда я перекрашиваю B в красный и для всего поддерева из корня P черная высота уменьшается на 1.



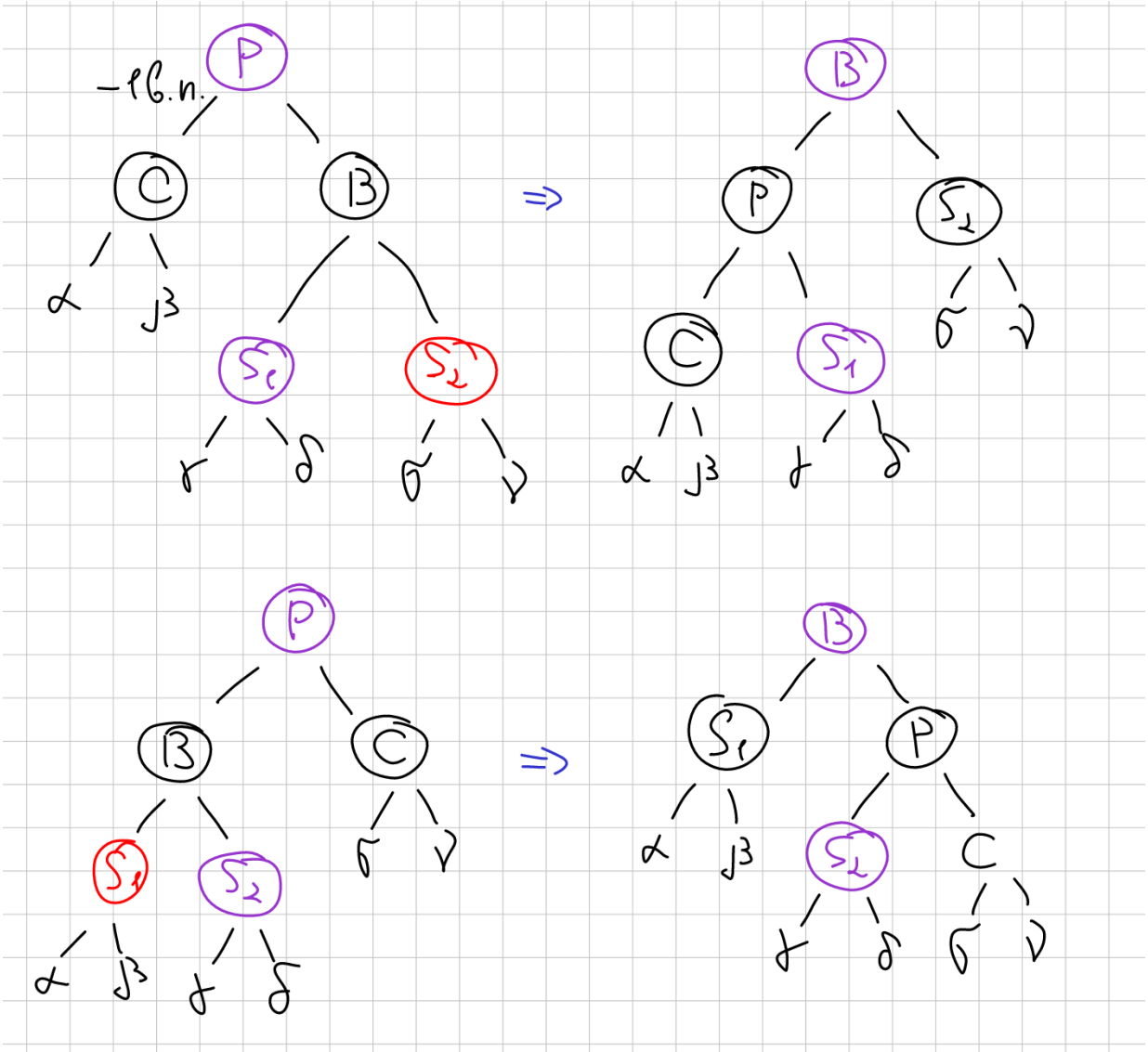
Пусть у P есть предок D и брат E. Тогда $bh(P) + 1 = bh(E)$, следовательно, необходимо из C перейти в P и запустить балансировку высот оттуда.

3. Если B чёрный, его внутренний потомок красный, а внешний потомок чёрный (возможно `nullptr`). В таком случае делается поворот относительно B и его внутренного узла.



Получается ситуация из п.5.4, когда внешний сын B - красный.

4. B - чёрный, его внешний потомок красный, внутренний потомок может быть любого цвета. В таком случае делается поворот относительно узлов B и P, причём внешний потомок B перекрашивается в красный, цвет внутренного потомка не изменяется.



Получается ситуация, что на стороне, где не хватало чёрной вершины, мы ей добавили. На стороне, где чёрных вершин было достаточно мы одну забрали, но перекрашиванием красной в чёрной вернули баланс.

Сложность: $O(h)$ - h - высота дерева
Доказательство: Найти элемент $\approx O(h)$. При удалении поиск минимального не больше $O(h)$, балансировка для одной вершины выполняется за константные перевороты/ перекрашивания, в худшем случае балансировки могут осуществляться от листа до корня, что составит h штук. Итого: $O(h + h + h) = O(3h) \approx O(h)$.

Высота КЧ дерева

Теорема: КЧ дерево с n внутренними узлами имеет высоту не более $2\log(n + 1)$
Доказательство: Поддерево узла x содержит минимум $2^{bh(x)} - 1$ внутренний узел. Если высота x равна 0, то узел является листом, его поддерево содержит 0 узлов. Если же x узел с положительной высотой и двумя потомками, тогда каждый потомок имеет чёрные высоты $bh(x)$ или $bh(x) - 1$. Высота узла x больше высоты его потомков, следовательно,

каждый потомок имеет минимум $2^{bh(x)-1}-1$ внутренних узлов. Значит, корень дерева имеет минимум $\left(2^{bh(x)-1}-1\right)+\left(2^{bh(x)-1}-1\right)+1=2^{bh(x)}-1$.
Пусть h - высота КЧ дерева, $root$ - корень дерева. Т.к. красные узлы не могут идти подряд, то минимум половина узлов на любом пути будут чёрные. Следовательно, $bh(root) \geq \frac{h}{2}$.
Тогда $n \geq 2^{h/2}-1$. Логарифмируем и получаем:

$$\log(n+1) \geq h/2, \quad \text{или} \quad h \leq 2\log(n+1)$$